

Gong to SFDC Draft Agent — Writeup

WRITEUP — Gong → Salesforce Draft Agent

Author: Gonzalo Insua · Role applied for: AI Builder, Tastewise Revenue AI Submission: May 2026
· Architecture version: v6 (final, two-agent w/ tool-using reviewer)

Framework choice — and what I rejected

Picked: n8n with two `langchain.agent` nodes orchestrating three deterministic sub-workflows as `ai_tool` connections.

Why n8n: the brief explicitly references the wine-estate workflow I demoed in the interview. n8n’s executor visualizes per-node tool calls in real time — that’s the artifact a reviewer can verify in the Loom. The agent’s tool-call timeline is *visible*, not buried in CLI logs. Same lens as the interview demo.

Rejected: Claude Agent SDK (Python). It would have given me direct control over `tool_choice` (n8n doesn’t expose it) and cleaner agent loop semantics. I started a pivot to it after Haiku exhibited “draft-only” behavior under n8n’s LangChain wrapper (refused to call tools, emitted JSON inline), but ultimately solved that problem architecturally — by adding a *second* agent (`review_agent`) whose prompt makes tool-use *structurally necessary* (no transcript visibility → can’t verify the draft without calling tools). The Python SDK would have been more elegant; n8n was more demo-able. Visual demonstrability won.

Rejected: LangGraph / CrewAI. Same reasoning loss as Python SDK plus framework overhead I didn’t need. The brief is a 2-transcript task, not a 50-state graph.

Model selection per step

Step	Model	Why
Extraction (<code>extraction_agent</code>)	Claude Haiku 4.5	Pure JSON drafting from a known schema. No tools attached. The job is “produce structured output” — a use case Haiku does cheaply and quickly. Cost: ~3K input tokens / OPP.

Step	Model	Why
Review (review_agent)	Claude Sonnet 4.5	This is the agent that has to <i>call tools, read results, decide approve/revise/abstain</i> . Haiku's tool-call adherence in n8n's LangChain wrapper was unreliable in testing (drafted JSON inline rather than invoking tools). Sonnet's tool-calling is materially stronger. We pay ~2× the per-call cost on this step but get reliable tool-using behavior — which is the take-home's #1 evaluation criterion.
Quote grounding	No LLM — deterministic JS	A substring match is a substring match; an LLM here is wasted budget and adds non-determinism to a check that should be 100% reproducible.
Schema validation	No LLM — deterministic JS	Type/enum/format checks against a fixed schema. Same reasoning.
Uncertainty check	No LLM — deterministic heuristic engine	Initially built as a Haiku reviewer. Rejected after observing rate-limit collapse (a <i>second</i> LLM in the critical path doubled ITPM consumption per OPP). The same failure modes the LLM was catching are characterizable as explicit rules (smalltalk_detected , vague_commitment , late_stage_no_amount , etc.). At zero token cost, sub-100ms latency, 100% auditable.

Rejected per step (in-flow): Opus is unjustified inside the agent flow itself. The hardest reasoning step (review) is *“compare these tool results to a rule table and emit one of three actions”* — Sonnet handles it. Opus would be cost without quality gain. If I ever needed Opus in-flow, it would be for a future step like *generating a coaching note from a failed-deal post-mortem*, not for structured-update validation.

Where Opus DID earn its keep — out-of-flow, build time. I used **Claude Opus 4.7** as the planning / architecture co-pilot during this build, via Claude Code with the n8n MCP server. Three concrete uses: 1. **Architecture iteration.** Six versions of this workflow, each a real refactor

(two-agent v1 → agent-loop tools → deterministic post-pipeline → two-agent v6 with structurally-required tool-use). Opus held the trade-off space across iterations and surfaced the failure modes that drove each pivot. 2. **Cross-referencing my existing n8n stack.** I have ~20 production workflows in the same instance (Vitagel WhatsApp agent, ML tools, QUARA contable). Opus read the Vitagel agent's `ai_tool` topology and the existing tool sub-workflows, then mapped that pattern to this task — meaning the architecture here mirrors patterns that already work in production for me, not n8n boilerplate. 3. **Migrating tool logic.** The three deterministic tools were partially built before this take-home. Opus helped port them — slim return payloads, pre-bind upstream inputs via `$fromAI` expressions, shift `uncertainty_check` from Haiku to deterministic heuristics — while preserving their sub-workflow contracts.

This is the production reasoning the brief asks about: *the model used for build-time orchestration is a different selection problem from the model used at run-time*. Opus at build time amortizes its cost across many runs; Sonnet at run-time gets metered per OPP. Right model for each layer.

How I'd evaluate this in production

Eval harness concept — implementable in ~3 hours, not done here:

1. **Golden-set fixtures.** N transcripts (~30) covering rich, thin, edge-case (mid-call hand-off, AE-only monologue, multi-stakeholder, foreign-language sprinkles), each paired with `{expected_action, expected_payload, expected_grounded_fields, expected_signal_strength}`.
 2. **Per-run metrics:**
 - **Abstain accuracy:** binary, did agent's `abstain` match the fixture's expected.
 - **Field-F1:** for non-abstain cases, weighted F1 across MEDDPICC + stage + amount + close_date + next_step. Per-field cost-weighted (mis-stating `economic_buyer` is worse than mis-stating `decision_process`).
 - **Grounding pass rate:** % of `reasoning_trace` claims that pass `quote_grounding`.
 - **Schema validity rate:** should be 100% if `schema_validator` is wired correctly.
 - **Agent-path adherence:** % of runs where `meta.path_taken == "review_agent"` (not the deterministic fallback). Drop in this number is an early warning that the LLM's tool-calling reliability is degrading.
 - **overrides_sonnet rate:** % of runs where `uncertainty_check` flipped the agent's decision. Rising rate = extraction is becoming overconfident.
 3. **CI integration.** Run on every prompt or model change. Block deploy if abstain accuracy drops, grounding rate falls below 95%, or agent-path adherence falls below 90%.
 4. **Production canary.** First 10% of real Gong calls go through both the agent and a "no-update" baseline. AE clicks "approve / edit / reject" on the agent's draft. We measure: time-to-commit, % fields edited, % drafts rejected entirely. See "Outcome measurement" below.
-

Failure modes I'd expect in the wild + mitigations

Failure mode	Mitigation
Hallucinated quotes (agent cites a quote that's not in the transcript)	<code>quote_grounding</code> substring match catches them. Failed fields get dropped from payload + <code>reasoning_trace</code> .
Bad citations (quote IS in the transcript but doesn't justify the claimed value — <i>"You'll loop in Ben for next Tuesday?"</i> cited as evidence Priya is the champion)	Caught in testing this submission. Mitigated via prompt instruction with bad/good examples. Long-term: add a value-presence check to <code>quote_grounding</code> (does the cited quote literally contain the claimed value?).
Optimism bias (extraction marks a thin call as rich because Sonnet wants to be helpful)	<code>uncertainty_check</code> heuristic vetoes via <code>overrides_sonnet</code> . Conservative by design — false abstains beat false updates. AE can always escalate from a logged touch; they can't unsend a wrong update to leadership.
Schema-echo edge case (LangChain agent returns the schema definition rather than values)	<code>extraction_post_normalizer</code> detects (<code>{type:'object', properties:{...}}</code>) and forces abstain with logged error. <code>uncertainty_check</code> also flags <code>schema_echo_detected</code> .
Rate-limit collapse (multiple LLM calls per OPP × multiple OPPs in a minute > tier ceiling)	Sequential <code>splitInBatches</code> loop with 65s <code>rate_limit_wait</code> for Sonnet's 30K ITPM tier. Pre-bound heavy tool inputs (<code>utterances</code> is bound from upstream, not agent-filled — saves ~7K tokens/OPP). Slim tool return payloads.
Agent skips tools and fabricates <code>tool_summary</code>	5-point <code>quality_score</code> in <code>review_post_normalizer</code> . Score < 4 → IF gate routes to deterministic fallback. The fabrication risk is the reason the fallback exists — and we log <code>meta.path_taken</code> per OPP for observability.
Tool-input schema rejects empty arrays (n8n bug: <code>evidence_list:[]</code> errors out the toolWorkflow input validator)	Agent's prompt branches: when <code>draft_abstain=true</code> , it skips <code>quote_grounding</code> entirely (nothing to ground in an abstain). Tool only invoked when there's actual evidence to verify.
Drift in transcript format (a real Gong dump has different timestamp formatting, speaker tags, etc.)	<code>transcript_reader</code> regex is the single point of dialect translation. Add fixtures for new formats; if regex misses, the parsing throws and the OPP becomes a tracked failure rather than silent garbage.
Empty / malformed input (Gong webhook fires with empty body, or <code>transcript_raw</code> is null)	IF node guard at workflow entry: short-circuits to a logged "no-op" output rather than letting the agent

Failure mode	Mitigation
	run on garbage. Same pattern as the abstain branch but earlier in the pipeline — fail fast, fail visible.
Run-time errors anywhere in the workflow (LLM API outage, sub-workflow exception, schema_validator throws on edge case)	Global error-handler workflow wired via n8n's <code>errorWorkflow</code> setting. On any unhandled error: capture node name, error message, <code>opportunity_id</code> (if reached), execution URL, and timestamp; POST to a <code>#sfdc-agent-alerts</code> Slack channel in real time. AE on call sees the failed OPP within seconds and can re-run manually or escalate. The error workflow is shared across all four workflows in this repo (set in <code>settings.errorWorkflow</code>).

Outcome measurement — audit → MVP → measure

Audit (week 1): instrument the *current* AE workflow. For 20 AEs over 2 weeks, log per call: time spent in Salesforce post-call (manual stopwatch + browser-extension), # fields touched, # fields with citation provided. Establish baseline `t_baseline` (target hypothesis: ~30 min/call per the brief).

MVP (weeks 2–4): roll out the agent to a 5-AE pilot. Their workflow becomes: agent emits draft → AE reviews in a Salesforce-side panel → clicks **approve / edit / reject**. Log per call: `t_review` (review-to-commit time), `pct_fields_edited`, `pct_drafts_rejected`, `signal_strength` from `uncertainty_check`. AEs keep a ≤1-line note on any reject (“agent made up X”, “missed Y”, “abstained when shouldn’t have”).

Measure (week 5+): - **Primary:** `t_review` median ≤ 90s per call (vs. 30 min baseline → 95% time reduction). Anything under 5 min is a meaningful win. - **Secondary:** `pct_drafts_rejected` ≤ 10%. `pct_fields_edited` ≤ 30% per accepted draft. - **Trust calibrator:** plot `confidence` vs. `pct_fields_edited` per draft. If high-confidence drafts have low edit rates, the model is well-calibrated; if not, recalibrate `confidence` formula. - **Abstain quality:** for every abstain, AE confirms “yes, no signal” or “no, you missed something.” Aim for ≥85% abstain accuracy.

Roll-out gate: if pilot hits primary metric for 2 consecutive weeks AND abstain accuracy ≥85%, expand to 25 AEs. Otherwise iterate on prompts/heuristics, re-baseline, re-pilot.

Decommission criterion: if `pct_drafts_rejected` > 25% for 2 consecutive weeks, kill the rollout, post-mortem, rebuild. Don’t let a half-trusted agent rot AE workflows.

What I cut, and what I’d add with another 4 hours

Cut for time: - Real eval harness (described above; not implemented). - Webhook trigger + Python CLI for programmatic invocation. Manual Trigger is fine for the demo. - Per-field

confidence (currently single payload-level confidence). Per-field would let an AE selectively edit low-confidence fields. - HTML output node for visual presentation (the brief says “no UI needed” and I respect “cut scope before you run over”). - **Hardened error handling**: input-guard IF node at workflow entry (short-circuits empty/malformed payloads) and a global `errorWorkflow` that POSTs node name + error + opportunity_id + execution URL + timestamp to a `#sfdc-agent-alerts` Slack channel in real time. The hooks are set (`settings.errorWorkflow` references it) — the channel-posting workflow itself is the one piece I haven’t wired in this submission. ~30 min of work to finish.

Would add with 4 more hours: 1. **Stricter `quote_grounding` with value-presence check.**

Substring match catches hallucinated quotes but not bad citations. A second pass that asks “does this quote literally contain the claimed value?” via a small LLM call (Haiku, cheap) would catch the citation-quality failures I observed in testing. 2. **Stricter quality threshold (6/8 dimensions instead of 4/5).** Current scoring is permissive. With more dimensions (parallel-tool-calls bonus, tool-call-order, evidence-citation-coverage, abstain-reasoning-depth) and a higher bar, we’d surface more agent failures earlier and route more borderline cases through the deterministic fallback. 3. **Eval harness with 30 fixtures.** The methodology is described above. Implementing it would let me cite a real adherence number (“Sonnet hits the agent path 94% of the time”) instead of a hand-wave. 4. **`overrides_sonnet` audit trail.** Every time the heuristic vetoes, log the decision with a payload diff so we can review whether the heuristic is conservative-but-right or too aggressive. 5. **Loom v2 with the eval harness running** showing 30 fixtures pass in CI before you’d ever ship a model-or-prompt change to production.